Week 3 - AWS SAM (templates, sam build / deploy, local testing)

DOP-C02 Domain 2 (Configuration Management and IaC). You must:

1. **Know when to choose SAM**: serverless apps (Lambda, API Gateway, DynamoDB, SQS, Step Functions) with **short IaC** and **local testing**.
2. **Read SAM templates**: Transform, Globals, `AWS::Serverless::*` resources, events, policy templates.
3. **Use the CLI**: `sam build`, `sam package`, `sam deploy`, `sam sync`, `sam pipeline`.
4. **Test locally**: `sam local invoke`, `start-api`, `start-lambda`, `generate-event` (needs **Docker**).
5. **Deploy safely**: **AutoPublishAlias + DeploymentPreference** (CodeDeploy canary / linear).

The big idea: **SAM = a CloudFormation transform + a CLI.** A SAM template is a CloudFormation template with `Transform: AWS::Serverless-2016-10-31`. CloudFormation expands the short resources and deploys a normal stack (change sets, rollback, drift and limits still apply).

Part 1 - SAM template anatomy

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31     # REQUIRED - makes it a SAM template
Description: Orders API

Globals:                                  # shared settings for all functions / APIs
  Function:
    Runtime: python3.12
    Timeout: 10
    MemorySize: 256
    Tracing: Active                       # X-Ray
    Environment:
      Variables:
        TABLE_NAME: !Ref OrdersTable

Parameters:
  Stage: { Type: String, Default: dev, AllowedValues: [dev, prod] }

Resources:
  OrdersFunction:
    Type: AWS::Serverless::Function
    Properties:
      CodeUri: src/orders/                # local folder, uploaded to S3 by package / deploy
      Handler: app.handler
      Policies:
        - DynamoDBCrudPolicy:             # SAM policy template (least privilege)
            TableName: !Ref OrdersTable
      AutoPublishAlias: live              # new version + alias "live" on each change
      DeploymentPreference:
        Type: Canary10Percent5Minutes
        Alarms: [!Ref ErrorsAlarm]
      Events:
        GetOrders:
          Type: Api                       # implicit REST API (ServerlessRestApi)
          Properties: { Path: /orders, Method: get }

  OrdersTable:
    Type: AWS::Serverless::SimpleTable    # DynamoDB, single primary key
    Properties:
      PrimaryKey: { Name: id, Type: String }

Outputs:
  ApiUrl:
    Value: !Sub "https://${ServerlessRestApi}.execute-api.${AWS::Region}.amazonaws.com/Prod/orders"
```

- Same sections as CloudFormation (Parameters, Mappings, Conditions, Resources, Outputs) **plus** `Globals`.
- **Mix SAM and plain CloudFormation resources** in the same template.

SAM resource types

Type	What it creates
`AWS::Serverless::Function`	Lambda function **+ IAM role** + event source mappings / permissions (+ version, alias, CodeDeploy if configured)
`AWS::Serverless::Api`	API Gateway **REST API** (explicit stages, auth, OpenAPI `DefinitionBody`, usage plans)
`AWS::Serverless::HttpApi`	API Gateway **HTTP API** (cheaper, simpler, JWT auth)
`AWS::Serverless::SimpleTable`	DynamoDB table with **one primary key**; use `AWS::DynamoDB::Table` for sort keys, GSIs, streams
`AWS::Serverless::LayerVersion`	Lambda layer (`ContentUri`, `CompatibleRuntimes`, `RetentionPolicy`)
`AWS::Serverless::StateMachine`	Step Functions (`DefinitionUri`, `DefinitionSubstitutions`, policy templates, events)
`AWS::Serverless::Application`	**Nested app** from the **Serverless Application Repository (SAR)** or S3
`AWS::Serverless::Connector`	**Permissions between resources** (Source, Destination, `Permissions: [Read, Write]`) without IAM JSON
`AWS::Serverless::GraphQLApi`	AppSync GraphQL API

Globals

- Sections: `Function`, `Api`, `HttpApi`, `SimpleTable`, `LayerVersion`, `StateMachine`.
- **Resource-level properties override Globals**; maps (env vars) are **merged**, lists are added.
- "Avoid repeating runtime, timeout and env vars in 20 functions" = **Globals**.

Function properties to know

Property	Notes
`CodeUri` / `InlineCode`	Local path (packaged to S3) or `s3://bucket/key`; inline for tiny functions
`Handler`, `Runtime`, `Architectures`	`x86_64` or `arm64`
`PackageType: Image`	`ImageUri` or `Metadata` (Dockerfile, DockerContext); `sam build` builds, `sam deploy` pushes to **ECR**
`Policies`	**SAM policy templates** (`S3ReadPolicy`, `DynamoDBCrudPolicy`, `SQSPollerPolicy`, `SNSPublishMessagePolicy`, `SSMParameterReadPolicy`, `AWSSecretsManagerGetSecretValuePolicy`...), managed policy names / ARNs, or inline statements
`Role`	Use an existing role (then `Policies` is ignored)
`AutoPublishAlias`	Publishes a **new version** on code / config change and points the **alias** at it
`DeploymentPreference`	**CodeDeploy** traffic shifting (needs AutoPublishAlias)
`ProvisionedConcurrencyConfig`	On the alias (needs AutoPublishAlias)
Others	`Layers`, `Environment`, `VpcConfig`, `Tracing`, `DeadLetterQueue`, `EventInvokeConfig`, `ReservedConcurrentExecutions`, `SnapStart`, `FunctionUrlConfig`
`Events`	Event sources (below)

Event source types

Type	Notes
`Api` / `HttpApi`	Without `RestApiId` SAM creates an **implicit API** (`ServerlessRestApi` / `ServerlessHttpApi`)
`S3`	**Bucket must be defined in the same template**

Type	Notes
`SQS`, `Kinesis`, `DynamoDB`, `MSK`, `MQ`, `SelfManagedKafka`, `DocumentDB`	**Event source mapping** (BatchSize, filtering, `ReportBatchItemFailures`)
`SNS`, `EventBridgeRule`, `Schedule`, `ScheduleV2`, `CloudWatchLogs`, `IoTRule`, `Cognito`	Invoke permissions created for you

What the transform generates

You write	CloudFormation gets
`AWS::Serverless::Function`	`AWS::Lambda::Function` + `<Name>Role` + permissions / event source mappings
+ `AutoPublishAlias: live`	`AWS::Lambda::Version` (`<Name>Version<hash>`) + `AWS::Lambda::Alias` (`<Name>Aliaslive`)
+ `DeploymentPreference`	`AWS::CodeDeploy::Application` (`ServerlessDeploymentApplication`) + **DeploymentGroup** + CodeDeploy role; alias gets **UpdatePolicy CodeDeployLambdaAliasUpdate**
`Events: Type: Api` (implicit)	`ServerlessRestApi` + Deployment + **Stage** `Prod` (plus an extra `Stage` stage, removed with `Globals: Api: OpenApiVersion: 3.0`)
`AWS::Serverless::SimpleTable`	`AWS::DynamoDB::Table`

Part 2 - DeploymentPreference (safe Lambda deployments)

```
AutoPublishAlias: live
DeploymentPreference:
  Type: Linear10PercentEvery1Minute   # Canary10Percent5/10/15/30Minutes, Linear..., AllAtOnce
  Alarms: [!Ref ErrorsAlarm, !Ref LatencyAlarm]   # alarm in ALARM = automatic rollback
  Hooks:
    PreTraffic: !Ref PreTrafficCheckFunction      # BEFORE traffic shifts
    PostTraffic: !Ref PostTrafficCheckFunction    # AFTER all traffic shifted
```

- **Requires** `AutoPublishAlias`**.**
- Hook functions **must call** `codedeploy:PutLifecycleEventHookExecutionStatus` (Succeeded / Failed), otherwise the deployment fails after the timeout. Names usually start with `CodeDeployHook_`.
- Alarm fires or hook fails = alias **moves back** to the old version.
- "10% for 5 minutes, automatic rollback on errors, minimal effort" = **AutoPublishAlias + DeploymentPreference Canary10Percent5Minutes + Alarms**.

Part 3 - The SAM CLI

```
sam init -> sam build -> sam local (test) -> sam deploy (--guided) -> sam logs -> sam delete
                         sam sync --watch (dev only, fast)
```

Command	What it does	Key points
`sam init`	Project from a template	`--runtime`, `--app-template`, `--package-type Image`
`sam validate`	Validates the template	`--lint` runs **cfn-lint**
`sam build`	**Resolves dependencies**, writes artifacts + **built template to** `.aws-sam/build/`	`--use-container` (Lambda-like Docker image, for **native dependencies**); `--cached`, `--parallel`; esbuild / makefile build methods; builds images
`sam package`	**Uploads artifacts to S3 (images to ECR)** and writes a **packaged template** with `s3://` URIs	Same as `aws cloudformation package`; `--output-template-file packaged.yaml`; `sam deploy` packages implicitly
`sam deploy`	Packages, then creates + executes a **CloudFormation change set**	`--guided` (saves `samconfig.toml`), `--resolve-s3`, `--capabilities CAPABILITY_IAM CAPABILITY_AUTO_EXPAND`, `--parameter-overrides`, `--no-confirm-changeset`, `--no-execute-changeset`, `--no-fail-on-empty-changeset`, `--config-env`, `--resolve-image-repos`
`sam sync`	**SAM Accelerate**: code changes go **straight to Lambda / API / Step Functions, bypassing CloudFormation**	`--watch`, `--code`; **dev only** (drift)
`sam logs` / `sam traces`	**CloudWatch Logs** / **X-Ray** traces	`-n Fn --stack-name app --tail`, `--filter`
`sam remote invoke`	Invoke a **deployed** function	`--event-file`
`sam remote test-event`	**Shareable test events** (EventBridge schema registry)	
`sam list`	endpoints / outputs / resources	
`sam publish`	Publish to the **Serverless Application Repository**	`Metadata: AWS::ServerlessRepo::Application`
`sam pipeline init --bootstrap`	Generates **CI/CD config** (CodePipeline, GitHub Actions, GitLab, Jenkins, Bitbucket) for **two stages**	`sam pipeline bootstrap` creates per stage: pipeline user / role, **CloudFormation execution role**, **artifact bucket**, **ECR repo**
`sam delete`	Delete the stack (and artifacts)	

samconfig.toml (per-environment settings)

```
version = 0.1
[default.deploy.parameters]
stack_name = "orders-dev"
resolve_s3 = true
capabilities = "CAPABILITY_IAM CAPABILITY_AUTO_EXPAND"
parameter_overrides = "Stage=dev"
confirm_changeset = true

[prod.deploy.parameters]
stack_name = "orders-prod"
parameter_overrides = "Stage=prod"
confirm_changeset = false
```

- `sam deploy --config-env prod` uses the prod settings. **Commit it**; **never store secrets** in it (use SSM / Secrets Manager).

Part 4 - Local testing

All `sam local` commands need Docker (containers that emulate the Lambda runtime).

Command	What it does	Use
`sam local invoke Fn -e event.json`	Runs **one function once** with an event	Debugging; `--env-vars env.json`; `-d <port>` debugger (AWS Toolkit); `--docker-network` (reach a local DB)
`sam local start-api`	**Local API Gateway on http://127.0.0.1:3000**	Test HTTP routes with curl; **code changes picked up without restart** (template changes need restart)
`sam local start-lambda`	Local **Lambda Invoke endpoint on port 3001**	**AWS SDK / CLI** with `--endpoint-url http://127.0.0.1:3001`, **automated integration tests**
`sam local generate-event`	**Sample event payloads**	`s3 put`, `sqs receive-message`, `apigateway aws-proxy`, `dynamodb update`, `sns notification`

What local testing does and doesn't emulate

- **Emulated**: runtime, handler, environment variables, timeout / memory, API Gateway routing (proxy).
- **Not emulated - IAM**: code runs with **your local credentials**. "Works locally, AccessDenied in AWS" = function role lacks permissions; fix `Policies`.
- **Not emulated - other AWS services**: DynamoDB, S3 calls go to **real AWS** unless you point the SDK at **DynamoDB Local** / LocalStack.
- **Not emulated - event source mappings** (SQS polling, Kinesis): simulate with `generate-event` + `invoke`.
- **Works with CDK**: `cdk synth`, then `sam local invoke -t cdk.out/MyStack.template.json MyFunction`. Terraform via `--hook-name terraform`.

Part 5 - SAM in CI/CD

Classic CodePipeline + CodeBuild pattern

```
# buildspec.yml
version: 0.2
phases:
  install:
    runtime-versions: { python: 3.12 }
  build:
    commands:
      - sam build
      - sam package --s3-bucket $ARTIFACT_BUCKET --output-template-file packaged.yaml
      # or: aws cloudformation package --template-file template.yaml
      #       --s3-bucket $ARTIFACT_BUCKET --output-template-file packaged.yaml
artifacts:
  files: [packaged.yaml]
```

- CodePipeline: **CloudFormation action CHANGE_SET_REPLACE**, optional **manual approval**, then **CHANGE_SET_EXECUTE**.
- Capabilities `CAPABILITY_IAM, CAPABILITY_AUTO_EXPAND`; template `BuildArtifact::packaged.yaml`; per-environment values via a **template configuration file** or parameter overrides.
- Alternatives: CodeBuild runs `sam deploy --no-confirm-changeset --no-fail-on-empty-changeset`, or `sam pipeline init --bootstrap`.
- **DeploymentPreference in the template** does the canary / linear shift when CloudFormation deploys - **no separate CodeDeploy action needed**.

Classic errors

Error	Fix
"Requires capabilities: [CAPABILITY_AUTO_EXPAND]"	Add **CAPABILITY_AUTO_EXPAND** (transform / nested apps)
Raw template with local `CodeUri: src/` in a CFN action	**Package first** (S3 URIs)
CI job fails when nothing changed	`--no-fail-on-empty-changeset`
"invalid ELF header" / missing .so at runtime	`sam build --use-container`

Part 6 - Serverless Application Repository and nested apps

```
AuthApp:
  Type: AWS::Serverless::Application
  Properties:
    Location:
      ApplicationId: arn:aws:serverlessrepo:us-east-1:123456789012:applications/auth
      SemanticVersion: 1.2.0
    Parameters: { UserPoolName: my-pool }
```

- Reuse published serverless apps, **shared privately** (accounts / org) or **publicly**.
- Needs `CAPABILITY_AUTO_EXPAND` (and `CAPABILITY_RESOURCE_POLICY` for some apps); becomes a **nested stack**.

Part 7 - SAM vs CloudFormation vs CDK

	CloudFormation	SAM	CDK
Format	YAML / JSON	**YAML + Serverless transform**	Programming language
Focus	Everything	**Serverless**	Everything (L2 / L3)
Local testing	No	`sam local`	Via SAM CLI on `cdk.out`
Gradual Lambda deploys	Manual CodeDeploy resources	**DeploymentPreference (one block)**	`LambdaDeploymentGroup`
Fast dev loop	No	`sam sync --watch`	`cdk watch` / hotswap
Keyword	Generic IaC	**Serverless, shorthand, local testing, Lambda canary**	Familiar programming language

Part 8 - Exam traps

Trap	Truth
SAM template without Transform	`Transform: AWS::Serverless-2016-10-31` **is required**
SAM deploys without CloudFormation	**SAM = CloudFormation transform**; change sets (except `sam sync` code updates)
Requires CAPABILITY_AUTO_EXPAND	Add it (plus CAPABILITY_IAM for roles)
CFN action deploys template with `CodeUri: ./src`	**Package first** (`sam package` / `aws cloudformation package`)
Native library fails after building on macOS / Windows	`sam build --use-container`
`sam local` says Docker is required	**Install and start Docker**
Works locally, AccessDenied in AWS	**Local uses your credentials**; fix the function `Policies`
`sam local` emulates DynamoDB / SQS	**No** - real AWS or DynamoDB Local; SQS polling not emulated
Test API routes locally	`sam local start-api` (3000)
SDK / integration tests against local functions	`sam local start-lambda` (3001)
Need a sample S3 / SQS event	`sam local generate-event`
Step-through debugging	`sam local invoke -d <port>` + IDE
Lambda canary in SAM	**AutoPublishAlias + DeploymentPreference** (Alarms, Hooks)
DeploymentPreference without AutoPublishAlias	**Invalid**
Hook never reports status	Must call **PutLifecycleEventHookExecutionStatus**
S3 event on a bucket from another stack	**Bucket must be in the same template**
SimpleTable with sort key / GSI	Use `AWS::DynamoDB::Table`
Repeated runtime / timeout	**Globals**
Least privilege without IAM JSON	**Policy templates** / **Connectors**
Fast code iteration in dev	`sam sync --watch` (dev only)
Different settings per environment	**samconfig.toml +** `--config-env`
Ready-made two-stage pipeline	`sam pipeline init --bootstrap`
Reuse an app across teams	**SAR +** `AWS::Serverless::Application`
Extra "Stage" stage on implicit API	`Globals: Api: OpenApiVersion: 3.0`

Part 9 - Hands-on lab (about 60 minutes, very low cost)

Prerequisites: SAM CLI, **Docker running**, AWS credentials, Python 3.12.

Lab 1 - Init and build

1. `sam init --runtime python3.12 --app-template hello-world --name sam-lab`, then `cd sam-lab`.
2. `sam validate --lint`, then `sam build`. Compare `template.yaml` (Transform, Globals, Events Api) with `.aws-sam/build/template.yaml` (CodeUri now points at the build folder).
3. Add a native dependency (for example `numpy`) to `requirements.txt` and run `sam build --use-container`.

Lab 2 - Local testing

1. `sam local generate-event apigateway aws-proxy --path hello --method GET > event.json`
2. `sam local invoke HelloWorldFunction -e event.json`
3. `sam local start-api`, then `curl http://127.0.0.1:3000/hello`. Edit the handler, save, curl again: new code without restart.
4. `sam local start-lambda`, then `aws lambda invoke --function-name HelloWorldFunction --endpoint-url http://127.0.0.1:3001 out.json`.
5. Create `env.json` with `{"HelloWorldFunction": {"GREETING": "hi"}}` and run `sam local invoke ... --env-vars env.json`.

Lab 3 - Deploy, logs, sync

1. `sam deploy --guided` (stack `sam-lab`, allow role creation, save to **samconfig.toml**). Read the change set preview.
2. In the CloudFormation console, view the **processed template**: generated Role, Permission, `ServerlessRestApi`, `Prod` stage.
3. Call the output URL, then `sam logs -n HelloWorldFunction --stack-name sam-lab --tail`.
4. `sam sync --watch --stack-name sam-lab`: edit code and see it update in seconds **without a change set**. Ctrl+C to stop.

Lab 4 - Canary deployment

1. Add `AutoPublishAlias: live` and `DeploymentPreference: { Type: Canary10Percent5Minutes, Alarms: [!Ref ErrorsAlarm] }`, plus a CloudWatch alarm on the function `Errors` metric.
2. `sam build && sam deploy`. Find the **CodeDeploy application and deployment group**, the version and the `live` alias.
3. Change the code and deploy again; in **CodeDeploy** watch 10% for 5 minutes, then 100%. Optional: make the new version throw errors and watch the **automatic rollback**.

Lab 5 - Pipeline (optional) and cleanup

1. `sam pipeline init --bootstrap` (CodePipeline or GitHub Actions). Review the generated file and bootstrap resources (roles, artifact bucket, ECR).
2. Cleanup: `sam delete --stack-name sam-lab`, delete pipeline bootstrap stacks, optionally `aws-sam-cli-managed-default`.

Part 10 - Check yourself

Questions

1. What line makes a CloudFormation template a SAM template, and what does it do?
2. What does a Function with AutoPublishAlias and DeploymentPreference create?
3. What does `sam build` do, and when do you need `--use-container`?
4. What does `sam package` / `aws cloudformation package` do, and why is it needed for a CodePipeline CloudFormation action?
5. A pipeline deploy fails with "Requires capabilities: [CAPABILITY_AUTO_EXPAND]". Fix?
6. What do `sam deploy --guided` and `samconfig.toml` give you? How do you deploy prod settings?
7. Difference between `sam local invoke`, `start-api` and `start-lambda`?
8. What must be installed for `sam local`?
9. Works with `sam local invoke` but AccessDenied in AWS. Why?
10. How do you get a realistic S3 put event for a local test?
11. What do you need for a 10% canary for 5 minutes with automatic rollback on errors?
12. What must a PreTraffic hook function do?
13. What is `sam sync --watch` for, and why not in prod?
14. When use `AWS::DynamoDB::Table` instead of SimpleTable? What are Globals for?
15. How do you quickly generate a two-stage pipeline, and what does bootstrap create?

Answers

1. `Transform: AWS::Serverless-2016-10-31`; CloudFormation **expands** `AWS::Serverless::*` into normal resources.
2. Lambda function, **IAM role**, permissions / mappings, **version**, **alias**, **CodeDeploy application + deployment group** + role; alias UpdatePolicy CodeDeployLambdaAliasUpdate.
3. **Resolves dependencies**, writes artifacts + built template to `.aws-sam/build`; `--use-container` for **native dependencies** built for Amazon Linux.
4. **Uploads code to S3** (images to ECR) and rewrites `CodeUri` to **S3 URIs**; CloudFormation can't read local paths.
5. Add **CAPABILITY_AUTO_EXPAND** (plus CAPABILITY_IAM).
6. Interactive first deploy that **saves settings to samconfig.toml**; `sam deploy --config-env prod`.
7. **invoke** = one function with an event; **start-api** = local HTTP API (3000); **start-lambda** = local Invoke endpoint (3001) for SDK / tests.
8. **Docker**.
9. Local runs use **your credentials**; the **function role lacks permissions** (fix Policies / Connector).
10. `sam local generate-event s3 put`.
11. **AutoPublishAlias** + **DeploymentPreference Canary10Percent5Minutes** + **Alarms**.
12. Run checks and call `PutLifecycleEventHookExecutionStatus` (Succeeded / Failed).
13. **Fast dev iteration**; it **bypasses CloudFormation** (drift, no change sets).
14. Sort key, GSIs, streams or advanced settings; **Globals** = shared properties for all functions / APIs.
15. `sam pipeline init --bootstrap`; per stage: pipeline user / roles, **CloudFormation execution role**, **artifact bucket**, **ECR repo**.

Quick memory summary

- **SAM = CloudFormation transform (**`AWS::Serverless-2016-10-31`**) + CLI** for serverless; Function, Api, HttpApi, SimpleTable, LayerVersion, StateMachine, Application, Connector; **Globals**; **policy templates**.
- **build** (`.aws-sam/build`, `--use-container`) -> **package** (S3 / ECR) -> **deploy** (change set, `--guided` / samconfig.toml, **CAPABILITY_IAM + CAPABILITY_AUTO_EXPAND**, `--no-fail-on-empty-changeset`).
- **Local testing needs Docker**: `invoke`, `start-api` (3000), `start-lambda` (3001), `generate-event`; **no IAM enforcement, no service emulation**.
- **Safe deploys**: AutoPublishAlias + DeploymentPreference (canary / linear, Alarms, hooks).
- **Dev speed**: `sam sync --watch` (dev only). **Pipelines**: `sam pipeline init --bootstrap` or CodeBuild `sam package` + CloudFormation action.